

Data Structures- Lists, Tuples, Dicts, Sets



Suman

22 Jan, 2026 At 9:00 PM

Pre-Reads

Trimester-1

Recommended

Resource



Associated Lectures (1)



Description



Discussions

SESSION 2.1 PRE-READ

Control Flow, Loops & Data Structures

Program: AIM101 --- Artificial Intelligence & Machine Learning Foundation **Pre-read Status:**
MANDATORY --- Must be completed before live session

Previous Sessions Recap: What You Already Know

Before Session 2.1, make sure you are comfortable with these concepts from Sessions 1.1 through 1.3:

Variables and Data Types

```
name = "Priya"      # str - text
age = 25             # int - whole number
height = 5.6         # float - decimal
is_student = True    # bool - True/False
```

Type Conversion

```
int("42")           # String to integer: 42
float("3.14")        # String to float: 3.14
str(100)             # Integer to string: "100"
```

Basic Operators

```
10 // 3              # Floor division: 3
10 % 3               # Modulo (remainder): 1
2 ** 3               # Exponentiation: 8
```

String Operations

```
text = "Python"
text[0]          # First character: "P"
text[-1]         # Last character: "n"
text[1:4]        # Slice: "yth"
text.upper()     # "PYTHON"
text.lower()     # "python"
```

Input/Output

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

What You Will Learn Today

By the end of Session 2.1, you will be able to:

Write `if / elif / else` statements to make decisions in your code

Create nested conditionals for complex decision trees

Use `for` loops with `range()` to repeat actions a specific number of times

Implement `while` loops for condition-based repetition

Control loop execution with `break` and `continue` statements

Create and manipulate lists for ordered, mutable collections

Work with tuples, dictionaries, and sets for different data organization needs

Choose the appropriate data structure for any given problem

The Business Story: The Data Organization Problem

Beyond Single Values

In Session 1.1, we stored a single student's name in a variable:

```
student_name = "Priya"
```

But what happens when your program needs to work with **real data**?

The Challenge

Imagine you are building a student management system for a college. Consider these questions:

Question 1: How would you store 1,000 student names?

Without data structures, you would need:

```
student1 = "Priya"
student2 = "Raj"
student3 = "Ananya"
# ... 997 more lines ...
student1000 = "Vikram"
```

This is unmanageable. What if you need to add a student? Delete a student? Find a specific student?

Question 2: How would you look up a student's grade by their name?

Without the right data structure:

```
if student_name == "Priya":
    grade = 85
elif student_name == "Raj":
    grade = 92
elif student_name == "Ananya":
    grade = 78
# ... hundreds more elif statements ...
```

This is unmaintainable and extremely slow.

The Solution: Data Structures

Data structures are specialized containers designed for different types of problems:

Problem	Data Structure	Why
Store 1,000 student names in order	List	Ordered, can add/remove items
Store coordinates that must not change	Tuple	Immutable, safe from accidental changes
Look up grades by student name	Dictionary	Key-value pairs for instant lookup
Track unique course enrollments	Set	Automatically removes duplicates

Real-World Impact

E-commerce Sites: When you browse Amazon, your shopping cart is a **list** (ordered items), product details are stored in **dictionaries** (name, price, description), and category filters use **sets** (unique categories only).

Social Media: Your friend list is a **list** (ordered by when you added them), your profile is a **dictionary** (name, email, bio), and mutual friends are calculated using **set** operations (intersection of two friend sets).

AI/ML Applications: Training data is stored in lists, model hyperparameters in dictionaries, unique labels in sets, and fixed configurations in tuples.

Today's Session: You will learn to choose and use the right data structure for any problem, combined with control flow and loops to process data efficiently.

Setup Requirements

Quick Environment Check (60 seconds)

Your environment should already be set up from previous sessions. Just verify:

Open Terminal/Command Prompt

Activate your environment:

```
conda activate mlenv
```

Your prompt should show `(mlenv)` at the beginning.

Check Python version:

```
python --version
```

Expected: `Python 3.12.x`

Launch Jupyter:

```
jupyter notebook
```

Expected: Browser opens with Jupyter interface

If any step fails, refer back to the Session 1.1 Pre-read for detailed troubleshooting instructions.

Concepts Preview

A. Conditionals: Making Decisions

Conditionals allow your program to make decisions based on conditions.

Basic if statement:

```
age = 20

if age >= 18:
    print("You are an adult")
```

if/else statement:

```
age = 15

if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

if/elif/else chain:

```
score = 75

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
else:
    print("Grade: F")
```

Key Rules:

- Colon `:` is required after each condition
- Indentation (4 spaces) defines the code block
- Only ONE branch executes in an if/elif/else chain

B. Nested Conditionals

When you need decisions within decisions:

```
has_license = True
age = 17

if age >= 18:
    if has_license:
        print("You can drive")
    else:
        print("Get a license first")
else:
    print("You must be 18 to drive")
```

C. For Loops: Repeating a Specific Number of Times**Using range():**

```
# Print numbers 0 through 4
for i in range(5):
```

```
print(i) # Prints: 0, 1, 2, 3, 4

# Print numbers 1 through 5
for i in range(1, 6):
    print(i) # Prints: 1, 2, 3, 4, 5

# Count by twos from 0 to 8
for i in range(0, 10, 2):
    print(i) # Prints: 0, 2, 4, 6, 8
```

The range() function:

- `range(stop)` - from 0 to stop-1
- `range(start, stop)` - from start to stop-1
- `range(start, stop, step)` - from start to stop-1, stepping by step

D. While Loops: Repeating Until a Condition Changes

While loops continue until their condition becomes `False` :

```
count = 1

while count <= 5:
    print(count)
    count = count + 1 # CRITICAL: must change the condition!

# Output: 1, 2, 3, 4, 5
```

Warning: If you forget to update the condition, you create an **infinite loop** that never stops!

E. Loop Control: break and continue

break - Exit the loop immediately:

```
for i in range(10):
    if i == 5:
        break # Stop the loop when i is 5
    print(i)

# Output: 0, 1, 2, 3, 4
```

continue - Skip to the next iteration:

```
for i in range(5):
    if i == 2:
        continue # Skip printing when i is 2
    print(i)

# Output: 0, 1, 3, 4
```

F. Lists: Ordered, Mutable Collections

Lists store multiple items in a specific order and can be changed:

```
# Creating a list
students = ["Priya", "Raj", "Ananya"]

# Accessing items (zero-indexed)
print(students[0])    # "Priya"
print(students[-1])   # "Ananya" (last item)

# Modifying items
students[1] = "Rahul" # Change "Raj" to "Rahul"

# Adding items
students.append("Vikram") # Add to end
students.insert(0, "Deepa") # Add at specific position

# Removing items
students.remove("Ananya") # Remove by value
deleted = students.pop()  # Remove and return last item

# List length
print(len(students))      # Number of items
```

G. Tuples: Ordered, Immutable Collections

Tuples are like lists but cannot be changed after creation:

```
# Creating a tuple
coordinates = (10, 20)
rgb_color = (255, 128, 0)

# Accessing items (same as lists)
print(coordinates[0]) # 10
print(coordinates[1]) # 20

# Cannot modify!
# coordinates[0] = 15 # ERROR! Tuples are immutable

# Unpacking tuples
x, y = coordinates
print(f"X: {x}, Y: {y}")
```

When to use tuples: When data should not change (coordinates, RGB values, database records).

H. Dictionaries: Key-Value Pairs

Dictionaries store data as key-value pairs for instant lookup:

```
# Creating a dictionary
student = {
    "name": "Priya",
    "age": 22,
    "grade": "A"
}
```

```
# Accessing values by key
print(student["name"])    # "Priya"
print(student.get("age")) # 22

# Adding/modifying entries
student["email"] = "priya@email.com" # Add new key
student["age"] = 23                  # Modify existing

# Removing entries
del student["grade"]              # Remove by key

# Check if key exists
if "name" in student:
    print("Name found!")

# Get all keys and values
print(student.keys())           # dict_keys(['name', 'age', 'email'])
print(student.values())         # dict_values(['Priya', 23, 'priya@email.com'])
```

I. Sets: Unique, Unordered Collections

Sets store only unique items with no duplicates:

```
# Creating a set
colors = {"red", "green", "blue"}

# Duplicates are automatically removed
numbers = {1, 2, 2, 3, 3, 3}
print(numbers) # {1, 2, 3}

# Adding items
colors.add("yellow")

# Removing items
colors.remove("red")

# Set operations
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

print(set_a | set_b)    # Union: {1, 2, 3, 4, 5, 6}
print(set_a & set_b)    # Intersection: {3, 4}
print(set_a - set_b)    # Difference: {1, 2}
```

When to use sets: Removing duplicates, membership testing, mathematical set operations.

Key Vocabulary

Familiarize yourself with these terms before class:

Control Flow Terms

Term	Definition
Condition	An expression that evaluates to True or False
Branch	One path of execution in an if/elif/else structure
Nested conditional	An if statement placed inside another if statement

Loop Terms

Term	Definition
Iteration	One complete pass through a loop
Loop	A code structure that repeats a block of code
Infinite loop	A loop that never stops (usually a bug!)
break	A keyword that exits a loop immediately
continue	A keyword that skips to the next iteration

Data Structure Terms

Term	Definition
Index	The position of an item in a sequence (starts at 0)
Slice	A portion of a sequence extracted using [start:stop]
Mutable	Can be changed after creation (lists, dicts, sets)
Immutable	Cannot be changed after creation (tuples, strings)
Key-value pair	A mapping from a key to its associated value (dictionaries)
CRUD	Create, Read, Update, Delete --- the four basic operations on data

Things to Ponder

Think about these questions before class. We will explore them during the session.

Why Do We Need Loops?

Without loops, how would you print "Hello" 100 times?

- What happens when requirements change to 1,000 times?
- How do loops make your code more maintainable?

Why Do Data Structures Matter?

If you have a list of 1 million items, how long does it take to check if an item exists?

- What if you used a set instead?
- Why is this difference important for real applications?

When would you choose a tuple over a list?

- What are the benefits of immutability?
- Can you think of data that should never change?

Design Decisions

Why does Python use zero-based indexing (starting at 0 instead of 1)?

- What are the advantages of this approach?
- How does slicing benefit from this?

Why are dictionary keys required to be immutable?

- What would happen if you could change a key after using it?
- Why does this rule exist?

Pre-Class Checklist

Before coming to class, ensure you have completed the following:

- ☐ Read this entire document
- ☐ Reviewed concepts from Sessions 1.1 through 1.3
- ☐ Verified your Python environment works (`conda activate mlenv`)
- ☐ Launched Jupyter Notebook successfully
- ☐ Understood the difference between lists, tuples, dictionaries, and sets
- ☐ Thought about the "Things to Ponder" questions
- ☐ Practiced the preview code examples in a Jupyter notebook (optional but recommended)

Quick Reference: Data Structure Comparison

Feature	List	Tuple	Dictionary	Set
Ordered	Yes	Yes	Yes (Python 3.7+)	No
Mutable	Yes	No	Yes	Yes
Duplicates allowed	Yes	Yes	Keys: No, Values: Yes	No
Access method	Index <code>[0]</code>	Index <code>[0]</code>	Key <code>["name"]</code>	N/A

Feature	List	Tuple	Dictionary	Set
Use case	Ordered collections	Fixed data	Key-value lookup	Unique items

See You in Class!

Today you will learn to organize data effectively and process it efficiently. These are foundational skills for every data science and machine learning application. Lists, dictionaries, and loops form the backbone of data processing in Python.

Get ready to work with real collections of data!

Vishlesan i-Hub IIT Patna x Masai School